PROJECT DELIVERY

# EdTech CRM & Inquiry System

Implementation, deployment, and handoff report

Prepared October 29, 2026

| DEPLOYED AND VERIFIED | Public frontend: Netlify \| API: Render \| Database: Supabase PostgreSQL |
|---|---|

## Project links

| LIVE DEMO | SOURCE REPOSITORY |
|---|---|
| stately-elf-d17f12.netlify.app | github.com/gamepatt78/Edtech-inquiry-crm |
| API HEALTH | PROJECT REPORT |
| edtech-inquiry-crm.onrender.com/api/health | documents/report.pdf |

## 1. Overview

The system collects education-related inquiries through a public form and gives staff a protected dashboard to review, filter, search, and update inquiry status. The browser app is deployed on Netlify; the Express API runs on Render and persists data in Supabase PostgreSQL.

## 2. Delivered functionality

• **Public inquiry form:** Parent, Student, and School user types; Scholarship, Concert, Admission, and General Query categories; contact details and message validation.

• **Phone validation:** Country selection and country-specific mobile-number length constraints in the form, with full international number validation in the API.

• **Staff dashboard:** Inquiry list, category and user-type filters, name/phone search, and status changes among New, In Progress, and Resolved.

• **Access control:** Custom backend login using bcrypt password hashes and signed JWTs; protected API routes require a valid Bearer token. The dashboard redirects unauthenticated visitors to Team login.

## 3. Architecture

| Layer | Implementation | Location |
|---|---|---|
| Frontend | React, Vite, React Router | frontend/ |
| Backend | Express API, validation, bcrypt, JWT | backend/ |
| Database | PostgreSQL tables, constraints, and indexes | schema.sql |
| Hosting | Netlify frontend; Render API; Supabase database | README.md |

## 4. Required assessment deliverables

| Requirement | Included | Notes |
|---|---|---|
| Clean full-stack repository | frontend/ and backend/ | Separate workspaces with application source and package manifests. |
| Database schema and SQL | schema.sql | Creates admins and inquiries tables, constraints, timestamps, and indexes. |
| Environment sample | .env.example | Documents DATABASE_URL, JWT settings, server options, and admin setup placeholders. No real credentials are included. |
| Setup and JWT testing guide | README.md | Covers prerequisites, local setup, admin creation, protected routes, and test commands. |
| Live demo | Netlify URL above | Production UI was opened and checked; API health returned HTTP 200 with status ok. |
| Supporting documents | docs/ and documents/ | Setup/login guides, inquiry examples, source assessment brief, and this report. |

## 5. Database and security notes

• **Schema:** `admins` stores unique administrator emails and password hashes. `inquiries` stores validated contact/query information and a constrained status. Database indexes support recent inquiry ordering and common category/user-type filters.

• **Credentials:** Copy `.env.example` to `.env` and supply private values locally or through the hosting provider. `.env` is ignored by Git; do not commit real database URLs, JWT secrets, or admin passwords.

• **JWT flow:** The API verifies credentials, issues a signed token, and checks both token validity and the associated administrator for protected requests. Login attempts are rate-limited.

## 6. Local setup and verification

• Install Node.js 20.19+ (or 22.12+) and copy `.env.example` to `.env`.

• Set `DATABASE_URL`, a random `JWT_SECRET` of at least 32 characters, and admin credentials; run `schema.sql` in the Supabase SQL Editor.

• From the repository root, run `npm install`, then `npm run create-admin --workspace backend`, then `npm run dev`.

• Open `http://localhost:5173`; sign in at `/admin`. To test the API guard, request `GET /api/inquiries` without a token and confirm HTTP 401.

**Verification performed:** Frontend production build succeeded. Backend test suite passed all 4 tests. The production API health endpoint returned `{"status":"ok"}`.

## 7. File organization

The root retains the GitHub-facing `README.md`, `.env.example`, and `schema.sql`. Supporting Markdown guides are in `docs/`; PDF deliverables are in `documents/`. Secrets in `.env` are excluded from version control.